

Università di Brescia

DIPARTIMENTO DI INGEGNERIA DELL' INFORMAZIONE

Corso di Laurea Magistrale

in INGEGNERIA INFORMATICA

ROBOT NEL GRID WORLD

Matteo Lublanis
Singh Sahiljit

736418, 740552

Indice

1. Definizioni - MDP, Bellman
2. Introduzione - Descrizione del problema
3. Programmazione dinamica
4. Pseudocodice: standard, in place
5. Analisi asintotica
6. Risultati sperimentali
7. Demo algoritmo
8. Conclusioni

Processo Decisionale di Markov - MDP

Un processo è detto markoviano quando la decisione dipende soltanto dallo stato corrente S dell'agente (robot in questo caso).

Ad ogni stato S, si associano delle azioni possibili che l'agente può scegliere. Di norma, ogni azione ha una probabilità di condurre allo stato successivo desiderato, ma non è detto: potrebbe cadere in uno stato peggiore.

Si utilizza anche una funzione di ricompensa (positiva o negativa) associata ad ogni azione, ove quella positiva è associata al goal finale mentre tutte le altre sono negative (anche influenzate dal passare del tempo.

Processo Decisionale di Markov - MDP

Nel nostro caso, si usano **ricompense additive** ma **senza sconto**. Di norma, nei MDP si utilizza un fattore di sconto γ compreso tra 0 e 1, elevato alla posizione dello stato nella sequenza decisionale:

$$U(s_0, s_1, \ldots, s_N) = \Upsilon^0 R(s_0) + \Upsilon^1 R(s_1) + \ldots + \Upsilon^N R(s_N)$$

Più è basso il valore, più importanza si dà alle ricompense immediate rispetto a quelle future.

Quindi un MDP è definito dalla quintupla: (S, A, P, R, γ).

Equazione di ottimalità di Bellman

L’equazione di Bellman è una funzione obiettivo definita in maniera ricorsiva, in aggiunta con la funzione di ricompensa scontata degli stati successivi.

$$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s,a) = \max_{a \in \mathcal{A}} \{R(s,a) + \gamma\, V^*(\delta(s,a))\}$$

Descrizione del problema

Sia dato un ambiente discreto descritto da una matrice NxN in cui ogni cella è o libera o un ostacolo per un Robot che deve muoversi da una cella di partenza S ad una cella obiettivo G.

Il Robot può tentare uno spostamento in una delle quattro direzioni cardinali (nord, sud, est, ovest) e può muoversi nella nuova cella se e soltanto se la cella è libera, altrimenti rimane fermo.

L'obiettivo è quello di determinare per ogni cella libera il valore della soluzione ottima V*, ovvero il premio R massimo (influenzato da numero di passi, urti contro gli ostacoli e raggiungimento obiettivo) ottenuto percorrendo il percorso minimo.

Descrizione del problema

- **S**: spazio degli stati, alias insieme di tutte le celle libere della matrice; ogni cella viene identificata da riga e colonna;
- **A**: spazio delle azioni, le quattro direzioni cardinali ammissibili per il robot;
- **δ**: funzione di transizione, ovvero lo spostamento nella nuova cella (se non è libera ritorna s iniziale);
- **R**: funzione di premio (diminuisce se aumenta il numero di passi o di urti contro ostacoli, aumenta di molto se si raggiunge il goal);
- **ɣ**: funzione di sconto, i premi futuri vengono valutati esattamente come quelli immediati, senza penalizzazione temporale

Descrizione del problema

- **Q***: funzione valore azione stato ottima, rappresenta il ritorno atteso eseguendo azione a nello stato s e proseguendo in modo ottimo da quel punto in avanti ($Q^*(s,a)=R(s,a)+\gamma V^*(\delta(s,a))$);
- **V***: funzione valore stato ottima, il massimo ritorno atteso scontato ottenibile a partire dallo stato s (se s=G allora $V^*(G)=0$, per ogni altro è il massimo di Q^*);
- **π***: politica ottima, funzione di decisione deterministica che associa ad ogni stato l'azione ottima da compiere (la tabella necessaria per la programmazione dinamica).

Programmazione Dinamica

Nella programmazione dinamica, un problema viene suddiviso in sottoproblemi, ogni sottoproblema viene risolto e la sua soluzione viene salvata in una tabella, la quale viene poi utilizzata per riprendere la stessa soluzione nel caso in cui si ripresenti lo stesso sottoproblema.

Per applicare questo metodo al nostro problema, bisogna dimostrare che è possibile strutturarlo in questa maniera. Il problema deve quindi rispettare due caratteristiche.

Programmazione Dinamica - Primo segno distintivo

Devono esistere sottostrutture ottime all'interno di una soluzione.

Sia P* un cammino ottimo che conduce dalla cella di partenza s_0 alla cella obiettivo G.

Se consideriamo una qualsiasi cella intermedia s_i facente parte del cammino ottimo di P*, il sottocammino P*_{i->k} rappresenta necessariamente la soluzione ottima al sottoproblema di raggiungere G partendo da s_i.

Se così non fosse, si contraddirebbe l'ottimalità di P*.

Programmazione Dinamica - Secondo segno distintivo

Tale algoritmo, nei diversi cammini del suo albero di ricorsione, può richiedere di risolvere più di una volta lo stesso sottoproblema.

$$V_k(s) = \max_{a \in \mathcal{A}} \{R(s,a) + \gamma\, V_{k-1}(\delta(s,a))\}\,, \qquad V_0 \equiv 0, \quad V_k(G) = 0.$$

Pseudocodice

Algoritmo 1 Aggiornamento di Bellman per una singola cella

BELLMAN-UPDATE(s, V_{old}, γ)

1: $max_q \leftarrow -\infty$; $best_a \leftarrow$ NIL
2: **for** $a \in \{\text{NORD}, \text{SUD}, \text{EST}, \text{OVEST}\}$
3: **do** $s' \leftarrow \delta(s, a)$
4: $q \leftarrow R(s, a) + \gamma \cdot V_{old}[s']$
5: **if** $q > max_q$
6: **then** $max_q \leftarrow q$; $best_a \leftarrow a$
7: **return** $(max_q, best_a)$

Pseudocodice

Algoritmo 2 Calcolo del valore di una soluzione ottima

GRID-VALUE-ITERATION$(Grid, N, G, \gamma, \epsilon)$
1: **for** $r \leftarrow 0$ **to** $N-1$
2: **do for** $c \leftarrow 0$ **to** $N-1$
3: **do** $V[r,c] \leftarrow 0$; $\pi[r,c] \leftarrow$ NIL
4: $\Delta \leftarrow \infty$; $k \leftarrow 0$; $max_iter \leftarrow N^2+1$
5: **while** $\Delta \geq \epsilon$ **and** $k < max_iter$
6: **do** $\Delta \leftarrow 0$; $k \leftarrow k+1$
7: $V_{old} \leftarrow V$
8: **for** $r \leftarrow 0$ **to** $N-1$
9: **do for** $c \leftarrow 0$ **to** $N-1$
10: **do** $s \leftarrow (r,c)$
11: **if** $Grid[s] \neq$ OSTACOLO **and** $s \neq G$
12: **then** $(V[s], \pi[s]) \leftarrow$ BELLMAN-UPDATE(s, V_{old}, γ)
13: $\Delta \leftarrow \max(\Delta, |V_{old}[s] - V[s]|)$
14: **if** $\Delta \geq \epsilon$
15: **then error** "Nessuna convergenza: esistono celle libere da cui G è irraggiungibile"
16: **return** (V, π)

Pseudocodice

Algoritmo 3 Costruzione di una soluzione ottima

CONSTRUCT-OPTIMAL-PATH$(S, G, \pi, Grid, N)$

1: $s \leftarrow S$; $P \leftarrow$ Queue(); Enqueue(P, s); $steps \leftarrow 0$
2: **while** $s \neq G$ **and** $steps < N^2$
3: **do** $a^* \leftarrow \pi[s]$
4: **if** $a^* =$ NIL **or** $\delta(s, a^*) = s$
5: **then error** "Mossa ottima non definita o diretta contro un ostacolo/bordo"
6: $s \leftarrow \delta(s, a^*)$
7: Enqueue(P, s); $steps \leftarrow steps + 1$
8: **if** $s \neq G$
9: **then error** "Goal non raggiungibile entro il numero massimo di passi consentiti"
10: **return** P

Pseudocodice - In Place

Algoritmo 4 Calcolo del valore di una soluzione ottima (variante in-place)

GRID-VALUE-ITERATION-IN-PLACE$(Grid, N, G, \gamma, \epsilon)$

1: **for** $r \leftarrow 0$ **to** $N-1$
2: **do for** $c \leftarrow 0$ **to** $N-1$
3: **do** $V[r,c] \leftarrow 0$; $\pi[r,c] \leftarrow$ NIL
4: $\Delta \leftarrow \infty$; $k \leftarrow 0$; $max_iter \leftarrow N^2+1$
5: **while** $\Delta \geq \epsilon$ **and** $k < max_iter$
6: **do** $\Delta \leftarrow 0$; $k \leftarrow k+1$
7: **for** $r \leftarrow 0$ **to** $N-1$
8: **do for** $c \leftarrow 0$ **to** $N-1$
9: **do** $s \leftarrow (r,c)$
10: **if** $Grid[s] \neq$ OSTACOLO **and** $s \neq G$
11: **then** $v_{old} \leftarrow V[s]$
12: $(V[s], \pi[s]) \leftarrow$ BELLMAN-UPDATE(s, V, γ)
13: $\Delta \leftarrow \max(\Delta, |v_{old} - V[s]|)$
14: **if** $\Delta \geq \epsilon$
15: **then error** "Nessuna convergenza: esistono celle libere da cui G è irraggiungibile"
16: **return** (V, π)

Analisi Asintotica

	Complessità Temporale	Complessità Spaziale
Grid Value Iteration	Θ(K*N^2)	Θ(N^2) {*}
Construct Optimal Path	Θ(L+1)	Θ(N^2)

Risultati sperimentali

Linguaggio usato: C++

Caratteristiche macchina:

- CPU: AMD Ryzen 7 5700X (8 core, 16 thread)
- RAM: 32 GB DDR4 3200MHz
- OS: Linux Mint

La valutazione delle prestazioni è stata condotta su un set di 10 istanze fisse caricate da file aventi dimensioni crescenti $N \in \{20, 25, 30, 40, 50, 60, 70, 80, 100, 120\}$, corrispondenti a griglie da 400 a 14.400 celle.

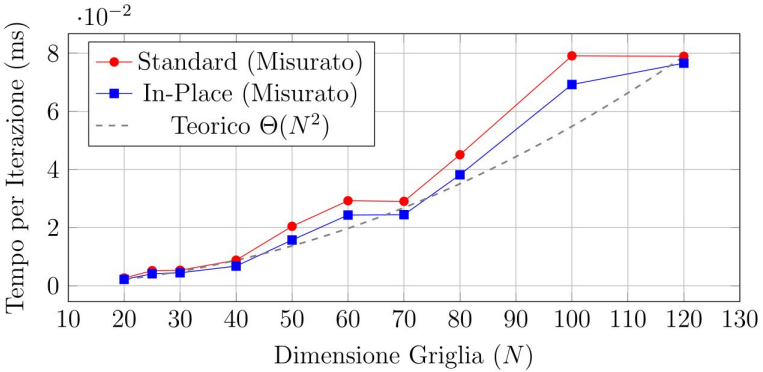
Risultati sperimentali

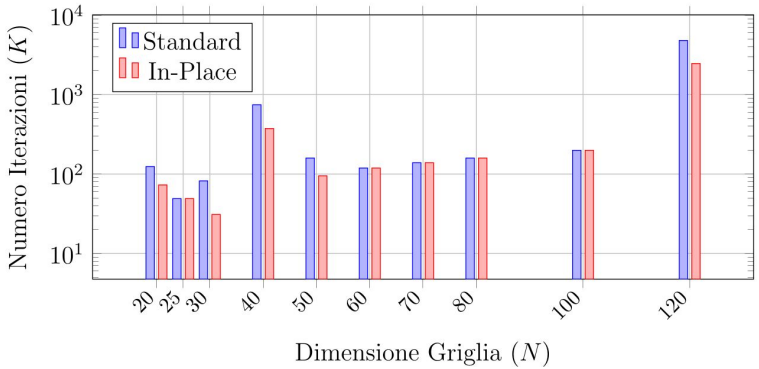
N	Variante	Celle (N^2)	Iterazioni	Tempo (ms)	Memoria Extra (KB)
20	Standard	400	124	0,328	3,59
	In-Place	400	73	0,159	0,00
25	Standard	625	49	0,254	5,47
	In-Place	625	49	0,202	0,00
30	Standard	900	82	0,437	7,73
	In-Place	900	31	0,138	0,00
40	Standard	1.600	745	6,551	13,44
	In-Place	1.600	374	2,524	0,00
50	Standard	2.500	159	3,251	20,70
	In-Place	2.500	95	1,494	0,00
60	Standard	3.600	119	3,480	29,53
	In-Place	3.600	119	2,894	0,00
70	Standard	4.900	139	4,036	39,92
	In-Place	4.900	139	3,398	0,00
80	Standard	6.400	159	7,166	51,88
	In-Place	6.400	159	6,070	0,00
100	Standard	10.000	199	15,742	80,47
	In-Place	10.000	199	13,778	0,00
120	Standard	14.400	4802	379,140	115,31
	In-Place	14.400	2462	188,524	0,00

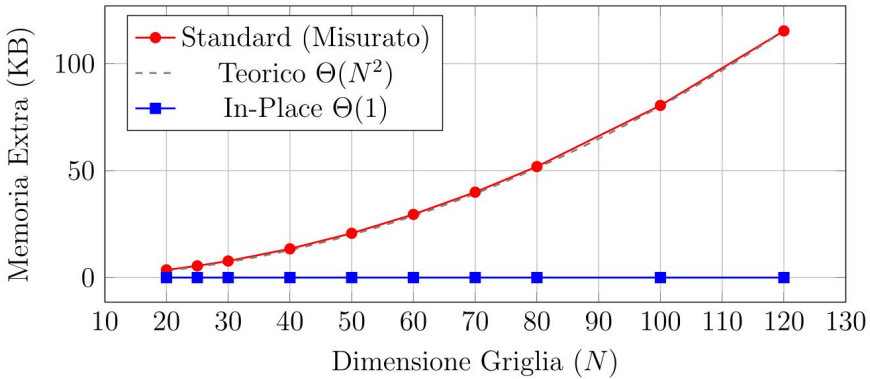
Risultati sperimentali

N	Celle (N^2)	$\lvert\mathcal{S}\rvert$	$\mathbf{ecc}(G)$	$\mathbf{ecc}(G)+1$ **(teorico)**	$K_{\mathbf{Standard}}$ **(misurato)**
20	400	292	123	124	124
25	625	580	48	49	49
30	900	546	81	82	82
40	1.600	916	744	745	745
50	2.500	2.010	158	159	159
60	3.600	3.376	118	119	119
70	4.900	3.379	138	139	139
80	6.400	5.347	158	159	159
100	10.000	9.433	198	199	199
120	14.400	9.798	4.801	4.802	4.802

Risultati sperimentali







Demo algoritmo

Pausa per provare algoritmo, illustrare i dati scelti

Conclusioni

Confronto prestazioni: Standard vs. In-Place

- **Iterazioni (Standard):** Limitate dall'eccentricità del Goal. Limite teorico confermato a O(N^4).
- **Convergenza accelerata (In-Place):** Iterazioni dimezzate (-48,7%) quando i percorsi assecondano il verso di scansione.
- **Impatto degli ostacoli:** Muri e colli di bottiglia contrari alla scansione annullano il vantaggio sui cicli dell'In-Place.
- **Efficienza temporale:** In-Place sempre più rapido (evita la copia della matrice ausiliaria V_old).
- **Complessità temporale:** Confermata a per entrambe le varianti a Θ(N^2).
- **Complessità spaziale:** Standard richiede spazio extra quadratico O(N^2); In-Place ottimizzato a O(1).